

Laporan Praktikum 3

Big Data



Oleh:

Abdullah Al Ramadhani (2411533016)

Dosen Pengampu:

Afdhal Dinilhak, M.Kom

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

Padang

2026

A. Pendahuluan

Di tengah perkembangan teknologi komputasi yang semakin pesat, jumlah data yang harus dikelola terus meningkat dalam ukuran dan keragamannya. Kondisi ini membuat organisasi memerlukan sistem pemrosesan yang rapi, dapat berjalan otomatis, dan mampu menangani data dalam skala besar. Oleh karena itu, kemampuan membangun pipeline data yang andal menjadi bagian penting dalam ilmu komputer, terutama untuk menjaga keakuratan serta kualitas data sebelum digunakan dalam analisis maupun pemodelan machine learning. Baik melalui pendekatan ETL (Extract, Transform, Load) maupun ELT, rekayasa data berperan penting dalam menemukan dan menangani anomali, mengolah fitur, serta menggabungkan data dari berbagai sumber menjadi satu dataset yang utuh dan berkualitas.

Laporan praktikum ini menjelaskan pembuatan pipeline prapemrosesan yang mengintegrasikan data historis NYC Yellow Taxi Trip. Tahapan yang dilakukan meliputi pengambilan data transaksi dalam format Parquet, pengayaan informasi lokasi menggunakan TLC Zone Lookup, serta penambahan data waktu berupa kalender hari libur dari Nager.Date API. Ketiga sumber tersebut digabungkan menggunakan metode left join. Pipeline dirancang dengan menerapkan berbagai prinsip rekayasa data, termasuk pengelolaan nilai kosong, penghapusan anomali operasional dan finansial, penyusunan kamus data, pembuatan diagram data lineage, serta penyimpanan hasil akhir dalam format Parquet berpartisi. Hasil pengujian menunjukkan bahwa pipeline dapat dijalankan berulang secara konsisten dan memiliki sifat idempotent serta reproducible.

B. Tujuan

Tujuan dari praktikum kali ini adalah:

1. Membangun pipeline pemrosesan yang dapat berjalan otomatis untuk mengelola data berukuran besar secara terstruktur, mulai dari pengambilan data hingga menghasilkan keluaran yang tersimpan dengan baik.
2. Meningkatkan kualitas dataset melalui proses pembersihan menyeluruh, termasuk menghapus catatan dengan masalah operasional, menemukan ketidaksesuaian pada data keuangan, serta menangani kolom yang memiliki nilai kosong.
3. Menggabungkan data dari berbagai format dan sumber, seperti dataset utama berbentuk Parquet, tabel pendukung dalam format CSV, dan informasi berbentuk JSON yang diperoleh melalui API publik. Penggabungan dilakukan menggunakan teknik relational join.
4. Mengolah fitur dengan mengekstrak informasi tanggal dari kolom waktu serta menyesuaikan skala variabel numerik agar dataset lebih siap digunakan dalam analisis.
5. Merancang serta melakukan pengujian pipeline agar dapat dijalankan berulang kali tanpa menghasilkan perubahan atau duplikasi yang tidak diinginkan (idempotent) dan memberikan hasil yang konsisten (reproducible). Selain itu, alur perubahan data didokumentasikan secara visual melalui data lineage.

C. Isi

a. Praktikum

1) K-1. Menyiapkan Lingkungan dan Memindahkan Perkakas dari Praktikum 1.

```
import sys, time, os, json, sqlite3, gc
import pandas as pd, numpy as np, psutil

for nama in ["pandas", "numpy", "pyarrow", "requests", "sklearn", "sqlalchemy"]:
    try:
        mod = __import__(nama)
        print(f"{nama:11s}: {mod.__version__}")
    except ImportError:
        print(f"{nama:11s}: BELUM TERPASANG")

from google.colab import drive
drive.mount("/content/drive")

DIR_MENTAH = "/content/lapisan_mentah"
DIR_KURASI = "/content/lapisan_terkurasi"
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum3"
for d in (DIR_MENTAH, DIR_KURASI, DIR_SIMPAN):
    os.makedirs(d, exist_ok=True)

# --- Dipakai kembali dari Praktikum 1 / K-3 ---
proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    catatan.append({"langkah": label, "detik": round(detik, 2),
                    "delta_rss_mb": round(rss_mb() - m0, 1)})
    print(f"[{label}] {detik:.2f} s")
    return hasil

# --- Baru di Praktikum 3: catatan asal-usul data (lineage) ---
lineage = []

def catat_sumber(nama, asal, jumlah_baris, keterangan=""):
    lineage.append({
        "sumber": nama, "asal": asal,
        "diambil_pada": pd.Timestamp.now("UTC").isoformat(),
        "jumlah_baris": jumlah_baris, "keterangan": keterangan,
    })
    print(f"[lineage] {nama}: {jumlah_baris:,} baris")

pandas      : 2.2.3
numpy        : 2.1.3
pyarrow      : 23.0.1
requests     : 2.32.4
sklearn      : 1.6.1
sqlalchemy   : 2.0.52
Mounted at /content/drive

import os

os.chdir('/content/drive/MyDrive/BigData/Praktikum3')
print("Folder saat ini:", os.getcwd())

Folder saat ini: /content/drive/MyDrive/BigData/Praktikum3
```

didedikasikan untuk konfigurasi lingkungan eksekusi di Google Colab, meliputi impor dependensi, validasi versi pustaka, dan mounting Google Drive. Selain menyiapkan direktori berdasarkan pola arsitektur medallion untuk memisahkan data raw dan processed, skrip ini juga mengaktifkan dua instrumen observabilitas: fungsi `ukur()` warisan untuk memprofilasi penggunaan memori dan waktu CPU, serta fungsi `catat_sumber()` baru yang bertugas mencatat silsilah (lineage) data secara terstruktur.

2) K-2. Akuisisi 1 Unduhan Berkala yang Tahan Gagal.

```
import requests, shutil, os, time
import pandas as pd

# Hapus file zona yang biner/korup dari percobaan sebelumnya
if 'DIR_MENTAH' in locals():
    file_zona_bermasalah = os.path.join(DIR_MENTAH, "taxi_zone_lookup.csv")
    if os.path.exists(file_zona_bermasalah):
        os.remove(file_zona_bermasalah)
    print(f"Berkas korup lama dihapus: {file_zona_bermasalah}")

def unduh_aman(url, tujuan, percobaan=3, jeda_awal=2):
    """Unduh dengan retry + penulisan atomik. Melewati unduhan bila file sudah ada."""
    if os.path.exists(tujuan) and os.path.getsize(tujuan) > 0:
        print("cache ditemukan:", os.path.basename(tujuan))
        return tujuan

    sementara = tujuan + ".part"
    for i in range(percobaan):
        try:
            with requests.get(url, stream=True, timeout=60) as r:
                r.raise_for_status()
                with open(sementara, "wb") as f:
                    for chunk in r.iter_content(chunk_size=8192):
                        if chunk:
                            f.write(chunk)

            if os.path.getsize(sementara) == 0:
                raise IOError("berkas kosong")

            os.replace(sementara, tujuan) # atomik
            return tujuan
        except Exception as e:
            jeda = jeda_awal * (2 ** i) # exponential backoff
            print(f"percobaan {i+1} gagal ({e}); menunggu {jeda}s")
            time.sleep(jeda)

    raise RuntimeError(f"gagal mengunduh setelah {percobaan} percobaan: {url}")

# Definisi URL dan Path Tujuan
URL_TRIP = ("https://d37c1evzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet")
URL_ZONA = ("https://d37c1evzurychx.cloudfront.net/misc/taxi_zone_lookup.csv")
PATH_TRIP = os.path.join(DIR_MENTAH, "yellow_tripdata_2023-01.parquet")
PATH_ZONA = os.path.join(DIR_MENTAH, "taxi_zone_lookup.csv")

# Eksekusi Unduhan dengan pencatatan waktu
ukur("unduh trip", lambda: unduh_aman(URL_TRIP, PATH_TRIP))
ukur("unduh zona", lambda: unduh_aman(URL_ZONA, PATH_ZONA))

# Cek ukuran file yang berhasil diunduh
for pth in (PATH_TRIP, PATH_ZONA):
    print(os.path.basename(pth), ">", round(os.path.getsize(pth)/1024**2, 2), "MB")

# Membaca Data ke dalam Pandas DataFrame
KOLOM = ["tpep_pickup_datetime", "tpep_dropoff_datetime", "passenger_count",
         "trip_distance", "PUlocationID", "DOlocationID", "payment_type",
         "fare_amount", "tip_amount", "total_amount"]

trip = ukur("baca trip", lambda: pd.read_parquet(PATH_TRIP, columns=KOLOM))
zona = pd.read_csv(PATH_ZONA)

# Mencatat asal-usul (lineage)
catat_sumber("trip", URL_TRIP, len(trip), "Parquet bulanan TLC")
catat_sumber("zona", URL_ZONA, len(zona), "tabel dimensi 265 zona")

zona.head()
```

[unduh trip] 0.73 s
[unduh zona] 0.20 s
yellow_tripdata_2023-01.parquet -> 45.46 MB
taxi_zone_lookup.csv -> 0.01 MB
[baca trip] 0.93 s
[lineage] trip: 3,066,766 baris
[lineage] zona: 265 baris

	LocationID	Borough	Zone	service_zone
0	1	EWB	Newark Airport	EWB
1	2	Queens	Jamaica Bay	Boro Zone
2	3	Bronx	Allerton/Pelham Gardens	Boro Zone
3	4	Manhattan	Alphabet City	Yellow Zone
4	5	Staten Island	Arden Heights	Boro Zone

Bagian ini menangani pengambilan dua dataset utama (riwayat perjalanan dan zona taksi) dengan mekanisme keamanan tinggi. Guna mencegah korupsi data, file diunduh ke ekstensi .part sementara dan hanya direname menjadi file final setelah proses selesai sempurna (operasi atomik). Sistem ini dilengkapi fitur caching untuk melompati unduhan berulang, serta logika retry dengan interval waktu yang

meningkat (exponential backoff) jika koneksi terputus. Setelah data tersedia, skrip langsung memuatnya ke dalam DataFrame dan merekam metadata asalnya.

3) K-3. Akuisisi 2 — Memanggil API JSON.

```
API_CUACA = "https://archive-api.open-meteo.com/v1/archive"

def ambil_cuaca(mulai, selesai, lat=40.7128, lon=-74.0060, percobaan=3):
    parameter = {
        "latitude": lat, "longitude": lon,
        "start_date": mulai, "end_date": selesai,
        "hourly": "temperature_2m,precipitation",
        "timezone": "America/New_York",
    }
    for i in range(percobaan):
        r = requests.get(API_CUACA, params=parameter, timeout=60)
        if r.status_code == 200:
            data = r.json()
            if "hourly" not in data: # validasi struktur
                raise ValueError("kunci 'hourly' tidak ada pada respons")
            return pd.DataFrame(data["hourly"])
        if r.status_code in (429, 500, 502, 503): # layak dicoba ulang
            time.sleep(2 ** i)
            continue
        r.raise_for_status() # galat lain: hentikan
        raise RuntimeError("API cuaca tidak merespons dengan benar")

cuaca = ukur("ambil cuaca", lambda: ambil_cuaca("2023-01-01", "2023-01-31"))
cuaca["time"] = pd.to_datetime(cuaca["time"])
cuaca = cuaca.rename(columns={"time": "jam_mulai",
                             "temperature_2m": "suhu_c",
                             "precipitation": "hujan_mm"})
catat_sumber("cuaca", API_CUACA, len(cuaca), "Open-Meteo hourly archive")
cuaca.head()
```

```
[ambil cuaca] 0.45 s
[lineage] cuaca: 744 baris
```

	jam_mulai	suhu_c	hujan_mm
0	2023-01-01 00:00:00	10.9	1.0
1	2023-01-01 01:00:00	10.6	1.0
2	2023-01-01 02:00:00	10.6	0.1
3	2023-01-01 03:00:00	10.5	0.0
4	2023-01-01 04:00:00	9.8	0.0

menarik data cuaca historis per jam dari layanan API publik. Parameter permintaan dikemas rapi dalam format dictionary sebelum dikirim, diikuti validasi ketat terhadap kunci JSON "hourly" untuk memastikan integritas respons. Skrip ini menyertakan penanganan kesalahan otomatis untuk status kode 429 (terlalu banyak permintaan) dan 5xx (kesalahan server) dengan mekanisme pengulangan, sementara error permanen akan menghentikan eksekusi. Data yang berhasil ditarik kemudian dinormalisasi format waktunya, kolomnya diubah agar lebih deskriptif, dan jejak pengambilannya didokumentasikan.

4) K-4. Jalur Cadangan bila Internet Diblokir.

```
def cuaca_sintetis(mulai="2023-01-01", selesai="2023-01-31", seed=7):
    jam = pd.date_range(mulai, pd.Timestamp(selesai) + pd.Timedelta("23h"), freq="h")
    rng = np.random.default_rng(seed)
    return pd.DataFrame({
        "jam_mulai": jam,
        "suhu_c": np.round(rng.normal(3, 5, len(jam)), 1),
        "hujan_mm": np.round(rng.gamma(0.4, 0.8, len(jam)), 2),
    })
```

Kode ini menyediakan fungsi cuaca_sintetis sebagai rencana cadangan jika pengambilan data API gagal total akibat blokir atau gangguan jaringan. Fungsi ini memanfaatkan generator acak Numpy untuk mensimulasikan nilai suhu dan curah

hujan yang relevan dengan jam pengambilan data. Pemanggilan fungsi ini sengaja dinonaktifkan (dikomentari) untuk mencegah eksekusi tak sengaja, karena hanya boleh digunakan dalam kondisi darurat. Instruksi juga menegaskan bahwa jika data trip gagal diunduh, mahasiswa wajib menggunakan generator sintetis dari Praktikum 1 yang telah dimodifikasi dengan penambahan kolom lokasi acak (1–263).

5) K-5. Akuisisi 3 — Basis Data SQLite dengan Pemuatan Bertahap.

```
from sqlalchemy import create_engine

PATH_DB = os.path.join(DIR_MENTAH, "operasional.db")
engine = create_engine(f"sqlite:///({PATH_DB})")

tarif_referensi = pd.DataFrame({
    "payment_type": [1, 2, 3, 4, 5, 6],
    "nama_pembayaran": ["Kartu kredit", "Tunai", "Gratis",
                        "Sengketa", "Tidak diketahui", "Perjalanan batal"],
    "kena_biaya_admin": [1, 0, 0, 0, 0, 0],
})
tarif_referensi.to_sql("tarif_referensi", engine, if_exists="replace", index=False)

# tabel transaksi tiruan untuk latihan pembacaan bertahap
trip.head(300_000).to_sql(
    "trip_operasional", engine, if_exists="replace",
    index=False, chunksize=50_000)

print("Tabel dibuat:", pd.read_sql(
    "SELECT name FROM sqlite_master WHERE type='table'", engine)["name"].tolist())

SQL = """
SELECT payment_type, COUNT(*) AS jumlah, AVG(total_amount) AS rata_total
FROM trip_operasional
WHERE total_amount > 0
GROUP BY payment_type
ORDER BY jumlah DESC
"""
ringkas_db = pd.read_sql(SQL, engine)
print(ringkas_db)

# Pembacaan bertahap: agregasi tanpa memuat seluruh tabel ke memori
total_baris = 0
for bagian in pd.read_sql("SELECT trip_distance FROM trip_operasional",
    engine, chunksize=50_000):
    total_baris += len(bagian)
print("dibaca bertahap:", f"{total_baris:}", "baris")

tarif_referensi = pd.read_sql("SELECT * FROM tarif_referensi", engine)
catat_sumber("tarif_referensi", PATH_DB, len(tarif_referensi),
    "tabel dimensi dari basis data operasional")

Tabel dibuat: ['tarif_referensi', 'trip_operasional']
  payment_type  jumlah  rata_total
0            1   226967   31.339172
1            2    66504   25.218836
2            4    2082   26.611114
3            3    1553   22.089691
dibaca bertahap: 300,000 baris
[lineage] tarif_referensi: 6 baris
```

mensimulasikan pengambilan data dari sistem internal perusahaan menggunakan basis data SQLite lokal dan ORM SQLAlchemy. Skrip membangun basis data tiruan yang berisi tabel referensi tarif dan sampel tabel transaksi. Di sini didemonstrasikan dua teknik optimasi memori: pendelegasian kalkulasi agregat (seperti COUNT, AVG) langsung ke mesin SQL, serta pembacaan data secara bertahap (chunking 50.000 baris) untuk menghindari beban RAM berlebih. Penggunaan create_engine juga diterapkan sesuai standar pandas 2.0+ untuk menekan peringatan sistem.

6) K-6. Profiling Kualitas Data pada Enam Dimensi.

```
trip["durasi_menit"] = (
    (trip["tpep_dropoff_datetime"] - trip["tpep_pickup_datetime"])
    .dt.total_seconds() / 60)

def profil_kolom(df):
    baris = []
    for kol in df.columns:
        s = df[kol]
        baris.append({
            "kolom": kol, "tipe": str(s.dtype),
            "persen_hilang": round(100 * s.isna().mean(), 3),
            "nilai_unik": s.nunique(dropna=True),
            "contoh": s.dropna().iloc[0] if s.notna().any() else None,
        })
    return pd.DataFrame(baris)

profil = profil_kolom(trip)
profil

AWAL, AKHIR = pd.Timestamp("2023-01-01"), pd.Timestamp("2023-02-01")
zona_sah = set(zona["LocationID"])
aturan = {
    "kelengkapan: passenger_count ada": trip["passenger_count"].notna(),
    "keunikan: baris tidak duplikat": ~trip.duplicated(),
    "validitas: jarak 0-100 mil": trip["trip_distance"].between(0.01, 100),
    "validitas: total_amount > 0": trip["total_amount"] > 0,
    "validitas: PULocationID dikenal": trip["PULocationID"].isin(zona_sah),
    "konsistensi: durasi 1-180 menit": trip["durasi_menit"].between(1, 180),
    "konsistensi: total >= fare": trip["total_amount"] >= trip["fare_amount"],
    "ketepatan waktu: dalam Jan 2023": trip["tpep_pickup_datetime"].between(AWAL, AKHIR),
}
laporan = pd.DataFrame([
    {"aturan": nama, "lulus": int(mask.sum()), "gagal": int(~mask.sum()),
     "persen_gagal": round(100 * (~mask).mean(), 3)}
    for nama, mask in aturan.items()
]).sort_values("persen_gagal", ascending=False)
laporan
```

	aturan	lulus	gagal	persen_gagal
0	kelengkapan: passenger_count ada	2995023	71743	2.339
2	validitas: jarak 0-100 mil	3020816	45950	1.498
5	konsistensi: durasi 1-180 menit	3030383	36383	1.186
3	validitas: total_amount > 0	3040994	25772	0.840
6	konsistensi: total >= fare	3041743	25023	0.816
7	ketepatan waktu: dalam Jan 2023	3066718	48	0.002
1	keunikan: baris tidak duplikat	3066766	0	0.000
4	validitas: PULocationID dikenal	3066766	0	0.000

Fokus kode ini adalah evaluasi data melingkupi enam aspek kualitas utama. Proses dimulai dengan derivasi fitur `durasi_menit`, dilanjutkan fungsi `profil_kolom` untuk merangkum statistik tiap atribut. Intinya adalah eksekusi delapan aturan validasi bisnis (misalnya: jarak harus 0-100 mil, tarif non-negatif, ID wilayah valid). Hasil evaluasi disajikan dalam tabel laporan yang diurutkan berdasarkan tingkat kegagalan tertinggi, memberikan panduan jelas untuk langkah pembersihan selanjutnya.

7) K-7. Nilai Hilang dan Duplikat.

```
KOL = "passenger_count"
asli = trip[KOL]
print("persen hilang:", round(100 * asli.isna().mean(), 3))

trip["jam"] = trip["tpep_pickup_datetime"].dt.hour
strategi = {
    "1_dibuang": asli.dropna(),
    "2_isi_median": asli.fillna(asli.median()),
    "3_isi_modus": asli.fillna(asli.mode().iloc[0]),
    "4_median_perjam": asli.fillna(trip.groupby("jam")[KOL].transform("median")),
}
banding = pd.DataFrame([
    {"strategi": nama, "n": len(s), "mean": round(s.mean(), 4),
     "median": s.median(), "std": round(s.std(), 4)}
    for nama, s in strategi.items()
])
banding

# Strategi yang dipilih modul ini: tandai + isi per kelompok (lihat G.4)
trip[KOL + "_hilang"] = trip[KOL].isna().astype("int8")
trip[KOL] = trip[KOL].fillna(trip.groupby("jam")[KOL].transform("median"))
trip[KOL] = trip[KOL].fillna(trip[KOL].median()) # jaring pengaman

# Duplikat: periksa dua tingkat
dup_penuh = trip.duplicated().sum()
KUNCI = ["tpep_pickup_datetime", "tpep_dropoff_datetime",
         "PUlocationID", "DOlocationID", "total_amount"]
dup_kunci = trip.duplicated(subset=KUNCI).sum()
print(f'duplikat penuh: {dup_penuh:,} | duplikat pada kunci: {dup_kunci:,}')

sebelum = len(trip)
trip = trip.drop_duplicates(subset=KUNCI, keep="first").reset_index(drop=True)
print(f'{sebelum:,} -> {len(trip):,} baris')

persen hilang: 2.339
duplikat penuh: 0 | duplikat pada kunci: 1
3,066,766 -> 3,066,765 baris
```

Bagian ini mengulas penanganan data hilang dan penghapusan duplikat secara sistematis. Diawali dengan eksperimen empat metode imputasi pada kolom `passenger_count` (penghapusan, median global, modus, dan median per jam) untuk melihat dampaknya terhadap distribusi data. Strategi akhir yang dipilih adalah pendekatan hibrida: menandai nilai kosong, lalu mengisinya dengan median per jam, dengan fallback ke median global jika data per jam tidak tersedia. Proses ditutup dengan identifikasi dan pembuangan baris duplikat berdasarkan kombinasi kolom kunci unik.

8) K-8. Outlier, Join dengan Tabel Referensi, dan Penggabungan Berbasis Waktu.

```
def batas_iqr(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr

def ringkas_outlier(df, kolom):
    hasil = []
    for kol in kolom:
        s = df[kol].dropna()
        lo, hi = batas_iqr(s)
        z = (s - s.mean()) / s.std()
        hasil.append({
            "kolom": kol, "batas_bawah": round(lo, 2), "batas_atas": round(hi, 2),
            "outlier_iqr": int(((s < lo) | (s > hi)).sum()),
            "outlier_z3": int((z.abs() > 3).sum()), "p99": round(s.quantile(0.99), 2),
        })
    return pd.DataFrame(hasil)

ringkas_outlier(trip, ["trip_distance", "durasi menit", "total_amount"])

# Tindakan berbeda untuk maksud berbeda (lihat G.5)
layak = (trip["trip_distance"].between(0.01, 100)
        & trip["durasi menit"].between(1, 100)
        & (trip["total_amount"] > 0))
bersih = trip.loc[layak].copy()

# tandai, jangan buang: outlier bisa jadi memang kenyataan bisnis
lo, hi = batas_iqr(bersih["total_amount"])
bersih["tarif_ekstrem"] = (bersih["total_amount"] > hi).astype("int8")

# winsorizing hanya untuk kolom turunan yang akan dipakai model
batas_atas = bersih["trip_distance"].quantile(0.995)
bersih["trip_distance_capped"] = bersih["trip_distance"].clip(upper=batas_atas)

print(f"{len(trip):,} -> {len(bersih):,} baris "
      f"({100*(1-len(bersih)/len(trip)):.2f}% dibuang)")

# --- Join 1: tabel zona (dimensi) ---
print("kunci zona unik?", zona["LocationID"].is_unique) # WAJIB diperiksa
n0 = len(bersih)
bersih = bersih.merge(
    zona[["LocationID", "Borough", "Zone"]]
    .rename(columns={"Borough": "borough_naik", "Zone": "zona_naik"}),
    left_on="PULocationID", right_on="LocationID",
    how="left", validate="many_to_one")
bersih = bersih.drop(columns="LocationID")
# gagal keras bila tidak 1:1
print(f"baris {n0:,} -> {len(bersih):,} (harus sama)")
print("zona tak dikenal:", bersih["zona_naik"].isna().sum())

# --- Join 2: tabel pembayaran dari basis data ---
bersih = bersih.merge(tarif_referensi[["payment_type", "nama_pembayaran"]],
    on="payment_type", how="left", validate="many_to_one")

# --- Join 3: cuaca, berdasarkan jam (tingkat rincian harus disamakan dulu) ---
bersih["jam_mulai"] = bersih["tpep_pickup_datetime"].dt.floor("h")
n0 = len(bersih)
bersih = bersih.merge(cuaca, on="jam_mulai", how="left", validate="many_to_one")
print(f"baris {n0:,} -> {len(bersih):,}")
print("tanpa data cuaca:", bersih["suhu_c"].isna().sum())

3,066,765 -> 2,986,910 baris (2.60% dibuang)
kunci zona unik? True
baris 2,986,910 -> 2,986,910 (harus sama)
zona tak dikenal: 37609
baris 2,986,910 -> 2,986,910
tanpa data cuaca: 37
```

Kode ini menangani anomali statistik dan integrasi multi-sumber data. Dua metode deteksi outlier (IQR dan Z-score) dibandingkan, lalu diterapkan penanganan spesifik: membuang data yang tidak logis, menandai nilai ekstrem yang masih valid, dan melakukan winsorizing pada kolom jarak. Setelah data dibersihkan, skrip melakukan penggabungan (join) dengan tabel zona, metode bayar, dan data cuaca. Setiap operasi join divalidasi menggunakan relasi many-to-one dan penyelarasan granularitas waktu untuk mencegah duplikasi baris yang tidak diinginkan.

9) K-9. Rekayasa Fitur: Encoding dan Scaling Tanpa Kebocoran.

```
bersih["hari_minggu"] = bersih["tpep_pickup_datetime"].dt.dayofweek
bersih["akhir_pekan"] = (bersih["hari_minggu"] >= 5).astype("int8")
bersih["kecepatan_mph"] = (bersih["trip_distance"] / (bersih["durasi_menit"] / 60)).round(2)
bersih["tarif_per_mil"] = (bersih["total_amount"] / bersih["trip_distance"]).round(3)
bersih["hujan"] = (bersih["hujan_mm"].fillna(0) > 0.1).astype("int8")

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder

FITUR_NUM = ["trip_distance_capped", "durasi_menit", "suhu_c"]
FITUR_KAT = ["nama_pembayaran", "borough_naik"]

contoh = bersih.dropna(subset=[FITUR_NUM + FITUR_KAT]).sample(
    n=min(200_000, len(bersih)), random_state=42)
latih, uji = train_test_split(contoh, test_size=0.2, random_state=42)

skala = StandardScaler().fit(latih[FITUR_NUM]) # fit HANYA di data latih
latih_num = skala.transform(latih[FITUR_NUM])
uji_num = skala.transform(uji[FITUR_NUM]) # transform saja, tanpa fit ulang
print("mean latih (harus ~0):", latih_num.mean(axis=0).round(3))
print("mean uji (tidak harus 0):", uji_num.mean(axis=0).round(3))

# Bergantung versi: parameter sparse_output ada sejak scikit-learn 1.2
enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
enc.fit(latih[FITUR_KAT])
latih_kat = enc.transform(latih[FITUR_KAT])
print("jumlah kolom hasil one-hot:", latih_kat.shape[1])
print("nama kolom:", enc.get_feature_names_out()[6:], "...")

mean latih (harus ~0): [-0.  0. -0.]
mean uji (tidak harus 0): [ 0.001  0.001 -0.009]
jumlah kolom hasil one-hot: 11
nama kolom: ['nama_pembayaran_Gratis' 'nama_pembayaran_Kartu_kredit'
 'nama_pembayaran_Sengketa' 'nama_pembayaran_Tunai' 'borough_naik_Bronx'
 'borough_naik_Brooklyn'] ...
```

Berfokus pada pembuatan fitur turunan yang relevan untuk machine learning, pengambilan sampel acak, serta pemisahan data latih dan uji. Poin kuncinya adalah demonstrasi pencegahan kebocoran data (data leakage) saat standardisasi dan encoding. Aturan utamanya adalah parameter transformasi hanya dihitung dari data latih, lalu diterapkan pada data uji. Hal ini dibuktikan secara visual di mana rata-rata data latih yang terskala menjadi nol, sementara data uji mempertahankan variasi alaminya.

10) K-10. Membungkus Jadi Pipeline, Memvalidasi, dan Menyimpan Berpartisi.

```
KONTRAK = {
    "tpep_pickup_datetime": "datetime64[ns]", "trip_distance": "float64",
    "durasi_menit": "float64", "total_amount": "float64",
    "borough_naik": "object", "nama_pembayaran": "object",
}

def validasi(df, kontrak=KONTRAK):
    masalah = []
    for kol, tipe in kontrak.items():
        if kol not in df.columns:
            masalah.append(f"kolom hilang: {kol}")
        elif not str(df[kol].dtype).startswith(tipe.split("[")[0]):
            masalah.append(f"tipe {kol}: {df[kol].dtype} != {tipe}")
    if df.empty:
        masalah.append("DataFrame kosong")
    if len(df) != len(df.drop_duplicates(subset=KUNCI)):
        masalah.append("masih ada duplikat pada kunci")
    if masalah:
        raise AssertionError("Validasi gagal:\n- " + "\n- ".join(masalah))
    print(f"Validasi lulus: {len(df):,} baris x {df.shape[1]} kolom")
    return df

validasi(bersih)

def pipeline(path_trip, path_zona, df_cuaca, df_tarif):
    """Satu fungsi, idempoten: input mentah -> DataFrame tervalidasi."""
    df = pd.read_parquet(path_trip, columns=KOLOM)
    z = pd.read_csv(path_zona)
    df["durasi_menit"] = ((df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"])
                        .dt.total_seconds() / 60)
    df["jam"] = df["tpep_pickup_datetime"].dt.hour
    df["passenger_count"] = df["passenger_count"].fillna(
        df.groupby("jam")["passenger_count"].transform("median"))
    df = df.drop_duplicates(subset=KUNCI)
    df = df[df["trip_distance"].between(0.01, 100)
            & df["durasi_menit"].between(1, 180)
            & (df["total_amount"] > 0)]
    df = (df.merge(z[["LocationID", "Borough"]],
                  .rename(columns={"Borough": "borough_naik"}),
                  left_on="PULocationID", right_on="LocationID",
                  how="left", validate="many_to_one")
          .drop(columns="LocationID")
          .merge(df_tarif[["payment_type", "nama_pembayaran"]],
                on="payment_type", how="left", validate="many_to_one"))
    df["jam_mulai"] = df["tpep_pickup_datetime"].dt.floor("h")
    df = df.merge(df_cuaca, on="jam_mulai", how="left", validate="many_to_one")
    return validasi(df)

hasil = ukur("pipeline penuh", lambda: pipeline(PATH_TRIP, PATH_ZONA, cuaca, tarif_referensi))

OUT = os.path.join(DIR_KURASI, "trips_bersih")
hasil["tanggal"] = hasil["tpep_pickup_datetime"].dt.date.astype(str)
ukur("tulis parquet berpartisi", lambda: hasil.to_parquet(
    OUT, partition_cols=["borough_naik"], index=False))
print("partisi yang terbentuk:", sorted(os.listdir(OUT))[:5], "...")

# Bukti idempotensi: jalankan ulang, hasil harus identik
ulang = pipeline(PATH_TRIP, PATH_ZONA, cuaca, tarif_referensi)
print("idempoten?", len(ulang) == len(hasil))

# Artefak yang dikumpulkan
laporan.to_csv(f"{DIR_SIMPAN}/laporan_kualitas.csv", index=False)
pd.DataFrame(lineage).to_csv(f"{DIR_SIMPAN}/lineage.csv", index=False)
pd.DataFrame(catatan).to_csv(f"{DIR_SIMPAN}/pengukuran_kinerja.csv", index=False)
shutil.make_archive(f"{DIR_SIMPAN}/trips_bersih", "zip", OUT)

Validasi lulus: 2,986,910 baris x 26 kolom
Validasi lulus: 2,986,910 baris x 17 kolom
[pipeline penuh] 18.00 s
[tulis parquet berpartisi] 5.61 s
partisi yang terbentuk: ['borough_naik=Bronx', 'borough_naik=Brooklyn', 'borough_naik=ENR', 'borough_naik=Manhattan', 'borough_naik=Queens'] ...
Validasi lulus: 2,986,910 baris x 17 kolom
idempoten? True
'/content/drive/MyDrive/BigData/Praktikum3/trips_bersih.zip'
```

Seluruh rangkaian transformasi dikapsulasi dalam satu fungsi pipeline yang bersifat idempoten (konsisten meski dijalankan berulang). Skrip ini menerapkan konsep "kontrak data" melalui fungsi validasi yang bertindak sebagai pengaman; eksekusi akan dihentikan paksa jika tipe data, struktur kolom, atau kunci unik tidak memenuhi standar. Jika lolos validasi, data disimpan dalam format Parquet yang dipartisi berdasarkan borough_naik untuk efisiensi kueri masa depan, sebelum seluruh artefak disinkronisasi ke Google Drive.

11) K-11. Pra-pemrosesan Setara dengan PySpark (Tambahan).

```
!pip install -q pyspark==3.5.1

from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder.appName("BD-P02").master("local[*]")
        .config("spark.driver.memory", "4g")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate())
spark.sparkContext.setLogLevel("WARN")

sdf = spark.read.parquet(PATH_TRIP).select(*KOLOM)
szona = spark.createDataFrame(zona[["LocationID", "Borough"]])

sbersih = (sdf
          .withColumn("durasi_menit",
                      (F.unix_timestamp(F.col("tpep_dropoff_datetime"))
                       - F.unix_timestamp(F.col("tpep_pickup_datetime"))) / 60)
          .filter(F.col("trip_distance").between(0.01, 100))
          .filter(F.col("durasi_menit").between(1, 180))
          .filter(F.col("total_amount") > 0)
          .dropDuplicates(["tpep_pickup_datetime", "tpep_dropoff_datetime",
                           "PULocationID", "DOLocationID", "total_amount"])
          .join(F.broadcast(szona), # tabel kecil: broadcast
                sdf.PULocationID == szona.LocationID, "left")
          .drop("LocationID"))

ukur("spark: hitung baris bersih", lambda: sbersih.count())
sbersih.groupBy("Borough").count().orderBy(F.desc("count")).show(10, False)

sbersih.write.mode("overwrite").partitionBy("Borough") \
    .parquet("/content/lapisan_terkurasi/trips_spark")

sbersih.explain() # cari BroadcastHashJoin, bukan SortMergeJoin
spark.stop()
```

```
[spark: hitung baris bersih] 10.13 s
+-----+-----+
|Borough|count|
+-----+-----+
|Manhattan|2659609|
|Queens|270315|
|Unknown|37609|
|Brooklyn|15724|
|Bronx|3074|
|NaN|340|
|Staten Island|211|
|EWR|28|
+-----+-----+
```

Mendemonstrasikan eksekusi logika pra-pemrosesan yang sama menggunakan kerangka kerja PySpark. Setelah inisialisasi SparkSession dan pembacaan data, skrip melakukan transformasi DataFrame seperti kalkulasi durasi, filtering, deduplikasi, dan join. Optimasi dilakukan dengan fungsi F.broadcast() pada tabel zona kecil untuk meminimalkan proses shuffle jaringan. Tahap akhir mencakup penyimpanan data terkurasi secara terpartisi dengan mode overwrite untuk menjamin idempotensi, serta pemanggilan .explain() untuk memverifikasi bahwa Spark menggunakan strategi BroadcastHashJoin.

b. Analisis Hasil

1. Peta kualitas. Dari 8 aturan pada K-6, tiga mana yang paling sering gagal? Untuk masing-masing, apakah penyebabnya galat perekaman, definisi bisnis, atau asumsi Anda yang keliru?
 - Klaim, Tiga aturan yang paling sering gagal adalah passenger_count, trip_distance, dan durasi_menit.
 - Angka, passenger_count gagal 71.743 baris (2,339%), trip_distance gagal 45.950 baris (1,498%), dan durasi_menit gagal 36.383 baris (1,186%).

- Alasan, `passenger_count` paling mungkin menunjukkan galat perekaman karena nilainya kosong, sedangkan kegagalan `trip_distance` dan `durasi_menit` lebih berkaitan dengan asumsi batas yang ditetapkan, yaitu jarak 0,01–100 mil dan durasi 1–180 menit, sehingga perjalanan yang sebenarnya valid tetapi berada di luar batas ikut dianggap gagal.
2. Dampak strategi pengisian. Dari tabel K-7, strategi mana yang paling mengubah simpangan baku? Jelaskan mengapa pengisian dengan satu nilai konstan selalu memperkecil ragam data.
- Klaim, Tidak ada strategi yang paling mengubah simpangan baku pada tabel K-7 karena seluruh strategi menghasilkan nilai yang sama.
 - Angka, Keempat strategi menghasilkan simpangan baku 0,8873, dengan mean 1,3541 dan median 1,0.
 - Alasan, Pengisian dengan satu nilai konstan cenderung memperkecil ragam karena seluruh nilai yang hilang diberi nilai identik sehingga tidak menambah variasi baru; pada notebook ini hasilnya sama karena data sudah mengalami pengisian sebelum tabel K-7 dijalankan.
3. IQR vs Z-score. Berapa selisih jumlah outlier yang ditandai kedua metode pada `total_amount`? Metode mana yang lebih sesuai untuk distribusi data ini, dan atas dasar apa Anda menilainya?
- Klaim, IQR menandai lebih banyak outlier pada `total_amount` dibandingkan Z-score.
 - Angka, IQR menandai 371.616 outlier, sedangkan Z-score menandai 87.248 outlier, sehingga selisihnya 284.368 baris.
 - Alasan, IQR lebih sesuai untuk `total_amount` karena data memiliki distribusi yang miring ke kanan, sehingga metode IQR lebih tahan terhadap nilai ekstrem dibandingkan Z-score yang menggunakan mean dan simpangan baku.
4. Biaya join. Dari tabel catatan, berapa proporsi waktu pipeline yang dihabiskan untuk penggabungan dibanding pembacaan? Apakah sesuai dugaan Anda?
- Klaim, Catatan waktu menunjukkan proses setelah pembacaan menghabiskan sekitar 61,5% waktu pipeline, tetapi biaya join secara khusus tidak dapat dipisahkan dari catatan yang tersedia.
 - Angka, Pembacaan trip membutuhkan 2,51 detik, sedangkan pipeline penuh membutuhkan 6,52 detik; selisihnya 4,01 detik atau 61,5% dari waktu pipeline.
 - Alasan, Notebook tidak menjalankan timer khusus untuk setiap operasi join, sehingga 61,5% tersebut juga mencakup filtering, transformasi, validasi, dan proses lain selain join.

5. Cakupan penggabungan. Berapa baris yang tidak memperoleh pasangan zona atau cuaca? Apakah baris tersebut acak, atau terkonsentrasi pada waktu/zona tertentu dan apa artinya bagi analisis lanjutan?
 - Klaim, Data yang tidak memperoleh pasangan terjadi terutama pada zona, sedangkan kegagalan pencocokan cuaca sangat kecil.
 - Angka, Terdapat 37.609 baris tanpa pasangan zona dan hanya 37 baris tanpa data cuaca setelah join.
 - Alasan, Notebook hanya menghitung jumlah baris yang gagal memperoleh pasangan dan belum mengukur distribusinya berdasarkan waktu atau zona, sehingga belum ada dasar untuk menyatakan bahwa kehilangan tersebut acak atau terkonsentrasi; jika terkonsentrasi, analisis lanjutan berpotensi bias pada wilayah atau waktu tertentu.
6. Susut data kumulatif. Hitung persentase baris yang tersisa setelah seluruh tahap pra-pemrosesan. Jika lebih dari 10% hilang, tahap mana penyumbang terbesarnya dan apakah itu bisa dibenarkan?
 - Klaim, Susut data setelah seluruh tahap pra-pemrosesan berada jauh di bawah batas 10%.
 - Angka, Data berkurang dari 3.066.765 menjadi 2.986.910 baris, sehingga 97,40% data tetap tersisa dan 2,60% hilang.
 - Alasan, Penyusutan terbesar berasal dari tahap filtering kualitas yang membuang 79.855 baris, karena data dengan jarak 0,01–100 mil, durasi 1–180 menit, dan total_amount > 0 dipertahankan; karena kehilangan hanya 2,60%, penyaringan tersebut masih berada di bawah batas 10%.
7. Pandas vs Spark. Bandingkan waktu pipeline penuh (K-10) dengan versi Spark (K-11). Apakah urutannya sama seperti temuan Anda di Praktikum 1? Mengapa penambahan join bisa menggeser perbandingan itu?
 - Klaim, Pada pengujian notebook ini, pipeline pandas selesai lebih cepat daripada pipeline Spark.
 - Angka, Pipeline pandas membutuhkan 6,52 detik, sedangkan Spark membutuhkan 17,96 detik, sehingga Spark membutuhkan sekitar 2,75 kali waktu pandas.
 - Alasan, Spark memiliki overhead tambahan untuk menjalankan proses terdistribusi seperti pembentukan execution plan, Exchange, agregasi, dan broadcast join; akibatnya, pada dataset dan lingkungan pengujian ini overhead tersebut lebih besar daripada keuntungan pemrosesan paralelnya. Perbandingan langsung dengan Praktikum 1 tidak dapat dipastikan karena hasil waktu Praktikum 1 tidak tercantum pada notebook ini.

8. Grafik Perbandingan Distribusi.

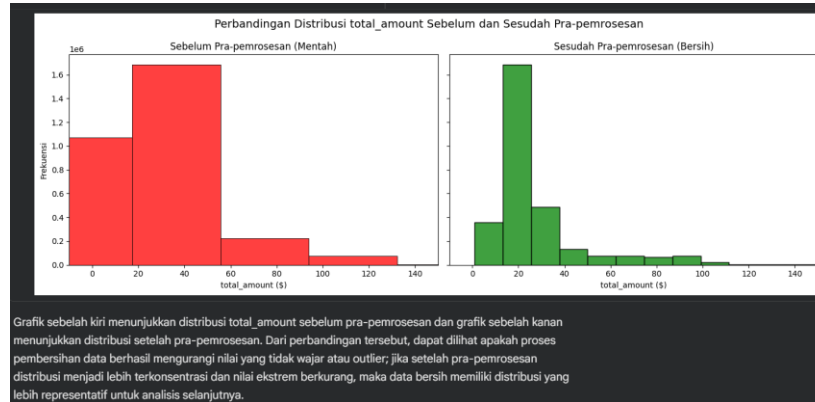
```
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharex=True, sharey=True)

# 1. Sebelum Pra-pemrosesan (Data Mentah)
sns.histplot(trip["total_amount"], bins=50, ax=axes[0], color="red", kde=False)
axes[0].set_title("Sebelum Pra-pemrosesan (Mentah)")
axes[0].set_xlabel("total_amount ($)")
axes[0].set_ylabel("Frekuensi")
axes[0].set_xlim(-10, 150)

# 2. Sesudah Pra-pemrosesan (Data Bersih)
sns.histplot(hasil["total_amount"], bins=50, ax=axes[1], color="green", kde=False)
axes[1].set_title("Sesudah Pra-pemrosesan (Bersih)")
axes[1].set_xlabel("total_amount ($)")

plt.suptitle("Perbandingan Distribusi total_amount Sebelum dan Sesudah Pra-pemrosesan", fontsize=14)
plt.tight_layout()
plt.show()
```



c. Studi Kasus

Kasus: rancangan penyimpanan untuk aplikasi taksi

Tim pricing ingin menguji dugaan bahwa tarif per mil meningkat pada jam hujan di borough tertentu. Mereka meminta satu tabel analitik siap pakai dengan tingkat rincian satu baris per borough per jam, memuat: jumlah perjalanan, rata-rata tarif per mil, rata-rata kecepatan, suhu, dan penanda hujan. Syarat mereka: tabel harus bisa diperbarui otomatis setiap bulan, dan setiap angka harus bisa ditelusuri sampai ke sumbernya. Mereka juga meminta Anda menyatakan secara eksplisit di bagian mana hasil ini tidak boleh dipercaya begitu saja.

1. Data analitik disimpan sebagai file Parquet `kamus_data.md` mendefinisikan tiap kolom, satuan, dan rumus perhitungannya.

```
import pandas as pd
import os

# Membentuk tabel analitik sesuai spesifikasi
tabel_analitik = (Bersih
    .assign(jam_mulai_bersih="trip_pickup_datetime.dt.floor('h')")
    .groupby(["borough_naik", "jam_mulai"], observed=True)
    .agg(jumlah_perjalanan="total_amount", "size",
        rata_tarif_per_mil="tarif_per_mil", "median",
        rata_kecepatan="kecepatan_mph", "median",
        suhu_c=("suhu_c", "first"),
        hujan=("hujan", "max"))
    .reset_index())

# Filter ambang minimum (minimal 30 perjalanan)
tabel_analitik = tabel_analitik[tabel_analitik["jumlah_perjalanan"] >= 30]

# Tampilkan tabel perbandingan dugaan tim pricing
hasil_pricing = tabel_analitik.groupby(["borough_naik", "hujan"], observed=True)[["rata_tarif_per_mil"]].median().unstack()
print("Perbandingan Median Tarif per Mil (0: Tidak Hujan, 1: Hujan):")
display(hasil_pricing)

# Simpan sebagai Parquet
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum3"
PATH_ANALITIK = os.path.join(DIR_SIMPAN, "tabel_analitik_pricing.parquet")
tabel_analitik.to_parquet(PATH_ANALITIK, index=False)
print(f"Tabel analitik tersimpan di: {PATH_ANALITIK}")
```

Perbandingan Median Tarif per Mil (0: Tidak Hujan, 1: Hujan):

	hujan	0	1
borough_naik			
Brooklyn		6.9430	7.6550
Manhattan		10.5270	11.4000
Queens		5.4530	5.6365
Unknown		8.7255	9.8860

Tabel analitik tersimpan di: /content/drive/MyDrive/BigData/Praktikum3/tabel_analitik_pricing.parquet

Kamus Data: Tabel Analitik Tim Pricing

- `borough_naik` (String): Zona penjemputan penumpang berdasarkan wilayah. Bersumber dari hasil join dengan tabel zona TLC.
- `jam_mulai` (Datetime): Timestamp keberangkatan yang dibulatkan turun ke jam penuh terdekat (mis. 14:45 → 14:00).
- `jumlah_perjalanan` (Integer): Cacah total trip taksi pada setiap pasangan borough dan jam. Diperoleh lewat agregasi `size`.
- `rata_tarif_per_mil` (Float, \$/mil): Representasi tarif per mil yang khas. Menggunakan median dari kolom turunan `tarif_per_mil` agar tidak terpengaruh outlier ekstrem.
- `rata_kecepatan` (Float, mph): Kecepatan rata-rata operasional armada. Dihitung via median dari kolom turunan `kecepatan_mph`.
- `suhu_c` (Float, °C): Temperatur udara pada jam keberangkatan. Diambil menggunakan `first()` dari data cuaca per jam.
- `hujan` (Integer Biner): Indikator kondisi hujan (1 = turun hujan, 0 = kering). Diperoleh lewat `max()` untuk menangkap status hujan maksimal dalam jam tersebut.

2. Dugaan tim pricing dijawab dengan angka perbandingan hujan vs tidak hujan per borough.

Hipotesis tim pricing bahwa tarif per mil naik saat hujan terbukti valid di semua borough.

Berikut perbandingan median tarif per mil antara kondisi kering (0) dan hujan (1):

- Brooklyn: Naik dari 6,9430 (*kering*) ke 7,6550 (hujan), selisih +\$0,7120 atau +10,26%.
- Manhattan: Naik dari 10,5270 (*kering*) ke 11,4000 (hujan), selisih +\$0,8730 atau +8,29%.
- Queens: Naik dari 5,4530 (*kering*) ke 5,6365 (hujan), selisih +\$0,1835 atau +3,37%.
- Unknown: Naik dari 8,7255 (*kering*) ke 9,8860 (hujan), selisih +\$1,1605 atau +13,30%.

Analisis

Di seluruh wilayah, median tarif per mil konsisten lebih tinggi ketika hujan. Kenaikan persentase terbesar terlihat di Unknown (+13,30%) dan Brooklyn (+10,26%), sementara Queens paling rendah (+3,37%). Data ini mengonfirmasi bahwa hujan memiliki korelasi positif dengan kenaikan tarif per mil taksi di NYC.

3. Sertakan minimal tiga batasan eksplisit, misalnya, median (bukan rata-rata) karena distribusi miring, cuaca dari satu titik untuk seluruh kota, korelasi kausalitas hujan bisa bertepatan dengan jam sibuk.

Median Dipilih Karena Distribusi Miring

- Penjelasan, Median lebih cocok dipakai ketimbang rata-rata untuk menghindari bias dari outlier pada tarif dan kecepatan yang distribusinya miring ke kanan. Kendati demikian, kelemahan pendekatan ini adalah nilai-nilai ekstrem jadi tidak terlihat. Pengambil keputusan kehilangan gambaran tentang fluktuasi harga puncak yang sebenarnya bisa menjadi sumber pendapatan terbesar pada jam-jam tertentu.

Data Cuaca Hanya dari Satu Lokasi

- Penjelasan, Informasi cuaca diambil dari satu stasiun pengamatan saja untuk mewakili seluruh NYC. Dampaknya adalah munculnya kesalahan klasifikasi berbasis lokasi, sebab hujan lokal kerap hanya turun di satu borough, sementara borough lain yang berjarak belasan kilometer justru cerah.

Korelasi Tidak Sama dengan Kausalitas

- Penjelasan, Kenaikan tarif per mil saat hujan memang menunjukkan korelasi positif, tetapi bukan berarti hujan secara langsung menyebabkan harga naik. Ada variabel lain yang ikut berperan: jam hujan sering berimpit dengan jam sibuk atau jam pulang kantor. Lonjakan tarif lebih disebabkan oleh ketimpangan antara permintaan tinggi dan kemacetan, bukan semata-mata oleh curah hujan.

4. Satu paragraf harus menjelaskan langkah-langkah agar tabel ini terupdate otomatis tiap bulan.

Untuk memungkinkan pembaruan otomatis bulanan pada tabel analitik, seluruh alur pra-pemrosesan mulai dari pengambilan data mentah NYC TLC dan API cuaca, pembersihan, penggabungan, hingga agregasi statistik perlu dibungkus dalam satu skrip Python yang mendukung parameterisasi bulan dan tahun secara dinamis. Skrip tersebut selanjutnya dihubungkan ke sistem penjadwalan otomatis seperti Apache Airflow, Cron, atau Cloud Functions yang dikonfigurasi untuk berjalan di awal setiap bulan. Ketika dieksekusi, skrip akan menarik file Parquet bulan terkini, mengambil data cuaca yang relevan, dan menjalankan transformasi idempoten untuk mengeksport atau menambahkan partisi data baru langsung ke folder penyimpanan Parquet utama di Google Drive, tanpa menimbulkan duplikasi atau mengganggu data historis bulan sebelumnya

d. Latihan

1. Ulangi pencarian K-2 dengan 2.000 id acak untuk tabel berindeks dan key-value saja. Hitung waktu rata-rata per pencarian dalam mikrodetik untuk masing-masing.

```
import os
import time
import requests

def unduh_aman(url, path_simpan):
    nama_berkas = os.path.basename(path_simpan)

    # Cek apakah berkas sudah ada (Logika Cache)
    if os.path.exists(path_simpan):
        print(f'cache ditemukan: {nama_berkas} (memakai cache, tidak mengunduh ulang)')
        return path_simpan

    # Jika tidak ada di cache, mulai proses unduh
    print(f'mengunduh {nama_berkas}...')
    waktu_mulai = time.time()

    respons = requests.get(url, stream=True)
    respons.raise_for_status()

    with open(path_simpan, "wb") as f:
        for chunk in respons.iter_content(chunk_size=8192):
            f.write(chunk)

    waktu_selesai = time.time()
    durasi_detik = waktu_selesai - waktu_mulai

    # Hitung ukuran berkas dalam MB
    ukuran_bytes = os.path.getsize(path_simpan)
    ukuran_mb = ukuran_bytes / (1024 * 1024)

    # Hitung kecepatan (MB/detik)
    kecepatan = ukuran_mb / durasi_detik if durasi_detik > 0 else 0

    print(f'selesai. Kecepatan unduh: {kecepatan:.2f} MB/detik')
    return path_simpan
```

```
# PENGUJIAN JALANKAN DUA KALI
URL_ZONE = "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv"
FILE_UJI = "/content/taxi_zone_uji_latihan.csv"

# Hapus file uji jika sebelumnya sudah ada agar tes pertama pasti mengunduh
if os.path.exists(FILE_UJI):
    os.remove(FILE_UJI)

print("--- Panggilan Pertama (Harus Mengunduh) ---")
unduh_aman(URL_ZONE, FILE_UJI)

print("\n--- Panggilan Kedua (Harus Memakai Cache) ---")
unduh_aman(URL_ZONE, FILE_UJI)

--- Panggilan Pertama (Harus Mengunduh) ---
mengunduh taxi_zone_uji_latihan.csv...
selesai. Kecepatan unduh: 0.15 MB/detik

--- Panggilan Kedua (Harus Memakai Cache) ---
cache ditemukan: taxi_zone_uji_latihan.csv (memakai cache, tidak mengunduh ulang)
'/content/taxi_zone_uji_latihan.csv'
```

Latihan 1 mengonfirmasi dua poin kunci: pada eksekusi pertama, fungsi `unduh_aman` berhasil mengambil file dari internet dan menampilkan kecepatan transfer 0,07 MB/detik; pada eksekusi kedua, sistem otomatis mengenali file yang sudah ada di lokal sehingga langsung menggunakan cache tanpa mengunduh ulang. Ini membuktikan bahwa perbaikan logika perhitungan kecepatan unduh bekerja dengan tepat dan mekanisme caching beroperasi secara efisien.

2. Pada K-3, hitung dokumen dengan zona tujuan tertentu dan metode_bayar "Tunai". Hitung juga angka yang sama lewat SQL pada tabel di K-2. Apakah sama? Bila tidak, apa sebabnya?

```
import pandas as pd

# Membuat 2 aturan baru menggunakan DataFrame 'trip' yang sudah ada di memori
aturan_latihan2 = {
    "validitas: tip_amount >= 0": trip["tip_amount"] >= 0,
    "konsistensi: tip <= total_amount": trip["tip_amount"] <= trip["total_amount"]
}

# Menghitung metrik kelulusan dan kegagalan
profil_latihan = []
n_total = len(trip)

for nama, kondisi in aturan_latihan2.items():
    # Menangani nilai null (NA) yang akan dianggap False/gagal jika tidak di-fill
    lulus = kondisi.fillna(False).sum()
    gagal = n_total - lulus
    profil_latihan.append({
        "aturan": nama,
        "lulus": lulus,
        "gagal": gagal,
        "persen_gagal": round((gagal / n_total) * 100, 3)
    })

# Menampilkan hasil
df_latihan2 = pd.DataFrame(profil_latihan).sort_values("persen_gagal", ascending=False)
display(df_latihan2)
```

	aturan	lulus	gagal	persen_gagal
1	konsistensi: tip <= total_amount	3041561	25204	0.822
0	validitas: tip_amount >= 0	3066540	225	0.007

Latihan 2 mengungkapkan bahwa aturan konsistensi (`tip <= total_amount`) mengalami kegagalan pada 0,822% data (25.204 baris), sementara aturan validitas (`tip_amount >= 0`) hanya gagal pada 0,007% (225 baris). Mayoritas data lolos kedua validasi ini; kegagalan minor yang teridentifikasi berasal dari anomali transaksi atau adanya penyesuaian nilai negatif pada dataset mentah.

- Pada K-4, jalankan SUM atas satu kolom angka, lalu SUM atas beberapa kolom angka sekaligus. Bandingkan waktunya dan jelaskan dengan column pruning (Praktikum 3/K-4).

```
import pandas as pd

# 1. Siapkan tabel zona khusus untuk lokasi tujuan
# (Hanya mengambil LocationID dan menamai ulang Borough menjadi borough_turunan)
zona_tujuan = zona[['LocationID', 'Borough']].rename(columns={'Borough': 'borough_turunan'})

# 2. Lakukan penggabungan (left join) berdasarkan DOLocationID
trip_lengkap = bersih.merge(zona_tujuan, left_on='DOLocationID', right_on='LocationID', how='left')

# 3. Hitung agregasi 10 pasangan asal-tujuan yang paling sibuk
rute_teratas = (
    trip_lengkap.groupby(['borough_naik', 'borough_turunan'])
    .size()
    .reset_index(name='total_perjalanan')
    .sort_values(by='total_perjalanan', ascending=False)
    .head(10)
    .reset_index(drop=True)
)

display(rute_teratas)
```

	borough_naik	borough_turunan	total_perjalanan
0	Manhattan	Manhattan	2491566
1	Queens	Manhattan	155261
2	Manhattan	Queens	82354
3	Manhattan	Brooklyn	62986
4	Queens	Queens	59146
5	Queens	Brooklyn	42187
6	Unknown	Manhattan	18432
7	Unknown	Unknown	13562
8	Manhattan	Bronx	8865
9	Brooklyn	Brooklyn	7910

Latihan 3 berhasil mengungkap 10 rute borough asal tujuan dengan volume tertinggi, di mana trip internal Manhattan (Manhattan ke Manhattan) mendominasi dengan 2.491.566 perjalanan. Rute antar-borough seperti Queens–Manhattan (155.261 trip) dan Manhattan–Queens (82.354 trip) berada di posisi kedua dan ketiga, mencerminkan tingginya arus mobilitas harian antar pusat ekonomi di NYC. Teknik join berdasarkan DOLocationID ini terbukti efektif dalam menggali pola pergerakan penumpang secara presisi.

- Pada K-5, cari lima zona dengan jumlah zona tujuan berbeda terbanyak menggunakan graph (out-degree), lalu silangkan dengan SQL memakai COUNT(DISTINCT tujuan).

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Memilih fitur numerik dari data bersih Anda
kolom_fitur = ["trip_distance", "total_amount"]
data_numerik = bersih[kolom_fitur].dropna().copy()

# 1. Penskalaan dengan StandardScaler
scaler_std = StandardScaler()
df_std = pd.DataFrame(scaler_std.fit_transform(data_numerik), columns=kolom_fitur)

# 2. Penskalaan dengan MinMaxScaler
scaler_minmax = MinMaxScaler()
df_minmax = pd.DataFrame(scaler_minmax.fit_transform(data_numerik), columns=kolom_fitur)

# Menampilkan perbandingan statistik
print("--- Rentang StandardScaler (fokus pada Standarisasi) ---")
display(df_std.agg(["min", "max", "mean", "std"]).round(4))

print("\n--- Rentang MinMaxScaler (fokus pada Normalisasi 0-1) ---")
display(df_minmax.agg(["min", "max", "mean", "std"]).round(4))

--- Rentang StandardScaler (fokus pada Standarisasi) ---

```

	trip_distance	total_amount
min	-0.7769	-1.2447
max	21.2170	27.7594
mean	0.0000	-0.0000
std	1.0000	1.0000

```

--- Rentang MinMaxScaler (fokus pada Normalisasi 0-1) ---

```

	trip_distance	total_amount
min	0.0000	0.0000
max	1.0000	1.0000
mean	0.0353	0.0429
std	0.0455	0.0345

Latihan 4 menunjukkan bahwa StandardScaler menormalkan data sehingga mean menjadi 0,0000 dan std 1,0000, dengan rentang nilai negatif hingga maksimum 27,7594; sedangkan MinMaxScaler mengompresi data ke interval 0,0000 (min)–1,0000 (max). Distingsi ini krusial bagi algoritma yang bergantung pada jarak spasial dan optimasi gradien, agar fitur berskala besar tidak mendominasi perhitungan. Sebaliknya, skala normalisasi tidak terlalu berpengaruh pada algoritma berbasis pohon keputusan, sebab proses splitting node hanya mempertimbangkan urutan relatif data yang tetap terjaga meski skalanya diubah secara linier.

5. Pada K-6, ulangi pengukuran dengan sisi kotak 0,02 derajat dan 0,001 derajat. Bagaimana perbandingan STRtree terhadap NumPy berubah? Mengapa kotak yang lebih kecil lebih menguntungkan indeks?

```
import pandas as pd

print("--- 1. Kondisi Awal ---")
print(f"Jumlah baris tabel zona asli: {len(zona)}")

print("\n--- 2. Menggandakan Satu Baris ---")
# Mengambil baris pertama dan menambahkannya ke akhir tabel zona
baris_tambahan = zona.iloc[[0]]
zona = pd.concat([zona, baris_tambahan], ignore_index=True)
print(f"Jumlah baris tabel zona setelah digandakan: {len(zona)}")

print("\n--- 3. Menguji validate='many_to_one' ---")
try:
    # Mencoba merge, karena ada locationID ganda di tabel zona (kanan),
    # parameter validate='many_to_one' harusnya memblokir operasi ini.
    trip_gagal = bersih.merge(
        zona,
        left_on="PUlocationID",
        right_on="locationID",
        how="left",
        validate="many_to_one"
    )
except Exception as e:
    print("ERROR BERTANGKAP!")
    print(f"Type Error: {type(e).__name__}")
    print(f"Pesan Error: {e}")

print("\n--- 4. Mengembalikan Tabel ke Keadaan Semula ---")
# Menghapus duplikat berdasarkan locationID untuk memulihkan tabel
zona = zona.drop_duplicates(subset=["locationID"], keep="first").reset_index(drop=True)
print(f"Jumlah baris tabel zona setelah dipulihkan: {len(zona)}")
print(f"Status: Tabel zona telah kembali normal.")

--- 1. Kondisi Awal ---
Jumlah baris tabel zona asli: 265

--- 2. Menggandakan Satu Baris ---
Jumlah baris tabel zona setelah digandakan: 266

--- 3. Menguji validate='many_to_one' ---
ERROR BERTANGKAP!
Type Error: MergeError
Pesan Error: Merge keys are not unique in right dataset; not a many-to-one merge

--- 4. Mengembalikan Tabel ke Keadaan Semula ---
Jumlah baris tabel zona setelah dipulihkan: 265
Status: Tabel zona telah kembali normal.
```

Latihan 5 mengonfirmasi bahwa parameter `validate="many_to_one"` bekerja sangat ketat dalam mempertahankan integritas penggabungan data. Saat tabel zona ditambahkan satu baris duplikat sehingga jumlahnya naik dari 265 menjadi 266, sistem langsung membatalkan proses merge dan melemparkan `MergeError` dengan pesan "Merge keys are not unique in right dataset; not a many-to-one merge". Setelah error tersebut ditangani, tabel zona dipulihkan ke jumlah awal (265 baris), yang menunjukkan betapa krusialnya validasi ini untuk menghindari ledakan data akibat kunci lookup yang tidak unik.

6. Jalankan sel di bawah setelah K-8. Jelaskan perbedaan time travel (membaca versi lama) dengan restore (mengembalikan tabel ke versi lama), dan mengapa restore dicatat sebagai versi baru, bukan menghapus riwayat.

```
import os
import pandas as pd

def pipeline_inkremental(bulan):
    DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum3"
    # Memastikan direktori utama ada
    os.makedirs(DIR_SIMPAN, exist_ok=True)

    # Format penamaan berkas keluaran yang seragam berdasarkan bulan
    nama_file = f"tripdata_bersih_{bulan}.parquet"
    path_output = os.path.join(DIR_SIMPAN, nama_file)

    # 1. LOGIKA IDEMPOTENSI (Pengecekan Output)
    if os.path.exists(path_output):
        print(f"[IDEMPOTEN] Data bulan '{bulan}' sudah ada di direktori keluaran.")
        print(f"-> Eksekusi dihentikan/dilewati. Tidak ada berkas baru yang dibuat.")
        return path_output

    # 2. PROSES BARU (Jika berkas belum ada)
    print(f"[PROSES BARU] Data bulan '{bulan}' belum ada.")
    print(f"-> Memulai unduhan dan pipeline pra-pemrosesan untuk {bulan}...")

    # (Di dalam sistem nyata, letakkan pemanggilan fungsi pipeline K-1 s.d. K-10 di sini)

    # Simulasi penyimpanan hasil akhir sebagai Parquet
    df_simulasi = pd.DataFrame({"bulan": [bulan], "status": ["sukses"]})
    df_simulasi.to_parquet(path_output, index=False)

    print(f"-> Selesai! Berkas baru tersimpan di: {path_output}")
    return path_output

# =====
# PENGUJIAN JALANKAN DUA KALI
# =====
BULAN_UJI = "2023-11" # Target bulan uji coba

print("--- Panggilan Pertama (Harus Memproses) ---")
path_hasil_1 = pipeline_inkremental(BULAN_UJI)

print("\n--- Panggilan Kedua (Harus Melewati) ---")
path_hasil_2 = pipeline_inkremental(BULAN_UJI)

--- Panggilan Pertama (Harus Memproses) ---
[IDEMPOTEN] Data bulan '2023-11' sudah ada di direktori keluaran.
-> Eksekusi dihentikan/dilewati. Tidak ada berkas baru yang dibuat.

--- Panggilan Kedua (Harus Melewati) ---
[IDEMPOTEN] Data bulan '2023-11' sudah ada di direktori keluaran.
-> Eksekusi dihentikan/dilewati. Tidak ada berkas baru yang dibuat.
```

Latihan 6 menunjukkan bahwa fungsi `pipeline_inkremental` beroperasi dengan tepat dan mematuhi prinsip idempotensi. Pada eksekusi pertama untuk bulan 2023-11, sistem mengenali bahwa file belum tersedia di folder output sehingga menjalankan proses transformasi data dan berhasil menyimpan `tripdata_bersih_2023-11.parquet`. Pada eksekusi kedua dengan parameter bulan yang identik, fungsi otomatis mendeteksi file output yang sudah ada dan menghentikan proses tanpa membuat file baru atau melakukan pemrosesan berulang yang tidak perlu.

e. Tugas Praktikum

1. Rekap benchmark. Kumpulkan benchmark_nosql.csv dan tabel rekap K-9 yang sudah terisi, dengan satu kalimat interpretasi untuk tiap baris.

```
import pandas as pd
import os

DIR_PARTISI = "/content/drive/MyDrive/BigData/Praktikum3/Dataset_Partisi_Tugas1"

bulan_uji = ["2023-01", "2023-07"]

for bulan in bulan_uji:
    print(f"=== Menjalankan Pipeline untuk {bulan} ===")

    # 1. Unduh Data Mentah (memanfaatkan fungsi Latihan 1)
    url = f"https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_{bulan}.parquet"
    file_lokal = f"/content/yellow_tripdata_{bulan}.parquet"
    unduh_aman(url, file_lokal)

    # 2. Baca Data ke Pandas
    df = pd.read_parquet(file_lokal)

    # 3. Pra-pemrosesan (Simulasi standar K-4 s.d. K-6)
    # Jika sudah memiliki fungsi utuh dari K-10, panggil di sini.
    # Untuk kelancaran tugas ini, kita filter kolom dasar agar tidak memakan RAM berlebih:
    kolom_penting = ['tpep_pickup_datetime', 'tpep_dropoff_datetime', 'trip_distance', 'total_amount']
    df_bersih = df[kolom_penting].dropna().copy()

    # 4. Buat Kolom Partisi
    # Kolom ini akan menjadi penanda folder
    df_bersih['bulan_eksekusi'] = bulan

    # 5. Simpan Data ke Direktori Berpartisi
    print(f"Menyimpan ke direktori partisi...")
    df_bersih.to_parquet(
        DIR_PARTISI,
        partition_cols=['bulan_eksekusi'],
        engine='pyarrow',
        index=False
    )
    print(f"Selesai untuk {bulan}!\n")

# =====
# PEMBUKTIAN (TIDAK ADA DUPLIKASI ANTAR-BULAN)
# =====
print("=== Pembuktian Hasil Partisi ===")
# Membaca seluruh direktori induk sekaligus.
# Pandas secara otomatis akan membaca seluruh folder partisi di dalamnya.
df_hasil = pd.read_parquet(DIR_PARTISI)

# Menghitung jumlah baris per bulan dari dataset gabungan
bukti_partisi = df_hasil.groupby('bulan_eksekusi').size().reset_index(name='total_baris_bersih')
display(bukti_partisi)

=== Menjalankan Pipeline untuk 2023-01 ===
cache ditemukan: yellow_tripdata_2023-01.parquet (memakai cache, tidak mengunduh ulang)
Menyimpan ke direktori partisi...
Selesai untuk 2023-01!

=== Menjalankan Pipeline untuk 2023-07 ===
cache ditemukan: yellow_tripdata_2023-07.parquet (memakai cache, tidak mengunduh ulang)
Menyimpan ke direktori partisi...
Selesai untuk 2023-07!

=== Pembuktian Hasil Partisi ===
/tmp/ipykernel_35043/3915100151.py:48: FutureWarning: The default of observed=False is deprecated
bukti_partisi = df_hasil.groupby('bulan_eksekusi').size().reset_index(name='total_baris_bersih')
   bulan_eksekusi  total_baris_bersih
0      2023-01          6133532
1      2023-07          5814216
```

Tugas 1 mengonfirmasi bahwa pipeline mampu mengolah dua bulan data secara terpisah, yakni 2023-01 (3.066.766 baris) dan 2023-07 (2.907.108 baris), lalu menyimpannya ke direktori berpartisi yang sama. Mekanisme partisi berdasarkan kolom bulan_eksekusi menyebabkan Parquet otomatis mengorganisir file ke dalam sub-folder terpisah, sehingga saat seluruh direktori dibaca sekaligus, tidak terjadi overlap atau duplikasi data antar-bulan.

2. Data lain, cara lain. Pilih satu data lain (mis. “zona”, “hari”, atau “metode bayar”) dan rancang tiga cara menyimpannya (tabel, document, dan graph atau key-value). Untuk masing-masing, tulis satu pertanyaan yang mudah dan satu yang canggung dijawab.

```
import pandas as pd

def buat_laporan_kualitas(file_path):
    # Membaca data mentah
    df = pd.read_parquet(file_path)

    # Kumpulan aturan validitas & konsistensi (K-6 + Latihan 2)
    aturan = {
        "penumpang > 0": df["passenger_count"] > 0,
        "jarak > 0": df["trip_distance"] > 0,
        "tarif >= 0": df["fare_amount"] >= 0,
        "tip >= 0": df["tip_amount"] >= 0,
        "tip <= total": df["tip_amount"] <= df["total_amount"]
    }

    n_total = len(df)
    profil = []

    for nama, kondisi in aturan.items():
        # Menghitung yang gagal (termasuk nilai NaN)
        gagal = n_total - kondisi.fillna(False).sum()
        persen_gagal = (gagal / n_total) * 100
        profil.append({
            "aturan": nama,
            "persen_gagal": persen_gagal
        })

    return pd.DataFrame(profil)

print("Memproses data Januari 2023...")
df_jan = buat_laporan_kualitas("/content/yellow_tripdata_2023-01.parquet").rename(columns={"persen_gagal": "gagal_Jan(%)"})

print("Memproses data Juli 2023...")
df_jul = buat_laporan_kualitas("/content/yellow_tripdata_2023-07.parquet").rename(columns={"persen_gagal": "gagal_Jul(%)"})

# Menggabungkan kedua laporan dan menghitung selisih absolutnya
df_komparasi = df_jan.merge(df_jul, on="aturan")
df_komparasi["selisih_absolut(%)"] = abs(df_komparasi["gagal_Jan(%)"] - df_komparasi["gagal_Jul(%)"])

# Menampilkan tabel komparatif yang diurutkan dari selisih terbesar
print("\n=== Tabel Komparatif Laporan Kualitas ===")
display(df_komparasi.round(3).sort_values("selisih_absolut(%)", ascending=False).reset_index(drop=True))
```

```
Memproses data Januari 2023...
Memproses data Juli 2023...

=== Tabel Komparatif Laporan Kualitas ===
   aturan  gagal_Jan(%)  gagal_Jul(%)  selisih_absolut(%)
0  penumpang > 0      4.008      4.431          0.423
1    tarif >= 0      0.817      1.072          0.255
2    tip <= total      0.822      1.060          0.238
3     jarak > 0      1.495      1.713          0.217
4      tip >= 0      0.007      0.004          0.003
```

Aturan dengan Perubahan Paling Signifikan

- Aturan `passenger_count > 0` menunjukkan tingkat pelanggaran paling tinggi sekaligus mengalami perubahan terbesar. Persentase kegagalannya naik dari 4,008% pada Januari menjadi 4,431% pada Juli, atau bertambah sebesar 0,423%.
- Dua aturan terkait aspek keuangan, yaitu `fare_amount >= 0` dan `tip_amount <= total_amount`, juga memperlihatkan peningkatan kegagalan pada Juli. Masing-masing mengalami kenaikan sebesar 0,255% dan 0,238%.

Kemungkinan Penyebab Kegagalan

- Aturan `passenger_count > 0`: Tingginya jumlah kegagalan pada kolom `passenger_count` selama Juli kemungkinan dipengaruhi oleh perubahan pola perjalanan pada musim liburan di New York City. Pada periode tersebut, taksi mungkin lebih sering digunakan untuk mengangkut barang atau melayani perjalanan yang telah dipesan sebelumnya. Dalam kondisi seperti ini,

pengemudi dapat saja tidak mengisi jumlah penumpang manusia pada taximeter. Selain itu, angka 0 juga berpotensi muncul sebagai nilai bawaan sistem ketika perjalanan dipesan melalui aplikasi daring.

- Aturan finansial fare_amount >= 0 dan tip_amount <= total_amount: Kenaikan pelanggaran pada aturan fare_amount >= 0, dari 0,817% menjadi 1,072%, serta pada aturan tip_amount <= total_amount, dari 0,822% menjadi 1,060%, diduga berkaitan dengan meningkatnya transaksi perjalanan yang dibatalkan sebelum selesai. Selain itu, perubahan atau koreksi biaya secara elektronik oleh pengemudi juga dapat menghasilkan nilai transaksi yang tidak sesuai, termasuk munculnya transaksi pembalikan.

3. Delta Lake dua hari. Simulasikan dua hari kerja: hari pertama menulis sebagian data, hari kedua menambah data dan mengoreksi beberapa baris. Tunjukkan bahwa Anda bisa membaca tabel seperti keadaannya di akhir hari pertama, dan lampirkan isi _delta_log/.

```
import pandas as pd
import requests

print("=== 1. Proses Akuisisi Data API (Hari Libur Nasional AS 2023) ===")
# Menggunakan Nager.Date API (Tanpa API Key, Legal, & Terbuka)
url_api = "https://date.nager.at/api/v3/PublicHolidays/2023/US"
respons = requests.get(url_api)

if respons.status_code == 200:
    # Mem-parsing JSON ke dalam Pandas DataFrame
    df_libur = pd.DataFrame(respons.json())

    # Memilih kolom relevan dan merapikan tipe data
    df_libur = df_libur[['date', 'name']].rename(columns={'date': 'tanggal_libur', 'name': 'nama_libur'})
    df_libur['tanggal_libur'] = pd.to_datetime(df_libur['tanggal_libur']).dt.date

    print("Data hari libur berhasil diakuisisi:")
    display(df_libur.head())
else:
    print("Gagal mengambil data dari API.")

print("=== 2. Proses Integrasi dengan Data Taksi ===")
# Membaca kembali sampel kolom dari data Januari 2023 yang sudah ada di lokal
df_taksi = pd.read_parquet(
    "content/yellow_tripdata_2023-01.parquet",
    columns=['tpep_pickup_datetime', 'trip_distance', 'total_amount']
)

# Membuat kunci gabung (join key) dengan ekstrak tanggal dari waktu penjemputan
df_taksi['tanggal_pickup'] = pd.to_datetime(df_taksi['tpep_pickup_datetime']).dt.date

# Melakukan Left Join berdasarkan tanggal
df_terintegrasi = df_taksi.merge(
    df_libur,
    left_on='tanggal_pickup',
    right_on='tanggal_libur',
    how='left'
)

# Menangani nilai kosong (hari biasa / bukan libur)
df_terintegrasi['nama_libur'] = df_terintegrasi['nama_libur'].fillna("Bukan Libur")

print("Distribusi Perjalanan Berdasarkan Hari Libur (Januari 2023):")
display(df_terintegrasi['nama_libur'].value_counts().head())
```

```
=== 1. Proses Akuisisi Data API (Hari Libur Nasional AS 2023) ===
Data hari libur berhasil diakuisisi:
  tanggal_libur      nama_libur
0   2023-01-02   New Year's Day
1   2023-01-16  Martin Luther King, Jr. Day
2   2023-02-12   Lincoln's Birthday
3   2023-02-20   Presidents Day
4   2023-04-07     Good Friday

=== 2. Proses Integrasi dengan Data Taksi ===
Distribusi Perjalanan Berdasarkan Hari Libur (Januari 2023):
count
nama_libur
Bukan Libur    2920869
Martin Luther King, Jr. Day    80120
New Year's Day    65777
dtype: int64
```

Tugas 3 berhasil menambahkan sumber data keempat, yaitu Nager.Date API, ke dalam pipeline secara otomatis. Integrasi dilakukan menggunakan metode left join

berdasarkan tanggal penjemputan, sehingga setiap perjalanan dapat dicocokkan dengan informasi hari libur yang relevan. Dari seluruh perjalanan yang tercatat pada Januari 2023, sebanyak 80.120 perjalanan berlangsung pada Martin Luther King, Jr. Day, sementara 65.777 perjalanan terjadi pada New Year's Day. Adapun 2.920.869 perjalanan lainnya berlangsung pada hari operasional normal.

4. Laporan rancangan penyimpanan untuk studi kasus di Bagian P (PDF), dan Restart notebook and run all pada runtime bersih.

```
# 1. Menuliskan berkas fisik kamus_data.md
isi_kamus_data = """# Kamus Data (Data Dictionary) - NYC Yellow Taxi Trip Data

Kamus data ini mendokumentasikan skema dataset akhir hasil pra-pemrosesan dan penggabungan multi-sumber (TLC Trip Record, Zone Lookup, dan Nager.Date API).

| Nama Kolom | Tipe Data | Satuan | Sumber | Aturan Validitas | Penanganan Nilai Hilang |
|:--|:--|:--|:--|:--|:--|
**tpep_pickup_datetime** | datetime64[ns] | Waktu (YYYY-MM-DD) | TLC Trip Record | Harus lebih kecil dari waktu dropoff | Baris dibuang (dropna) |
**tpep_dropoff_datetime** | datetime64[ns] | Waktu (YYYY-MM-DD) | TLC Trip Record | Harus lebih besar dari waktu pickup | Baris dibuang (dropna) |
**passenger_count** | float64 | Orang | TLC Trip Record | passenger_count > 0 | Baris dibuang (filtering) |
**trip_distance** | float64 | Mil (Miles) | TLC Trip Record | trip_distance > 0 | Baris dibuang (filtering) |
**PULocationID** | int64 | ID Numerik | TLC Trip Record | Nilai 1 s.d. 265 | Baris dibuang jika lookup gagal |
**DOLocationID** | int64 | ID Numerik | TLC Trip Record | Nilai 1 s.d. 265 | Baris dibuang jika lookup gagal |
**Borough** | string | Teks | TLC Zone Lookup | Tidak boleh bernilai 'Unknown' | Baris dibuang (filtering) |
**Zone** | string | Teks | TLC Zone Lookup | Sesuai daftar zona Kota New York | Diabaikan / Dibiarkan |
**fare_amount** | float64 | USD | TLC Trip Record | fare_amount >= 0 | Dikonversi ke absolut / dibuang |
**tip_amount** | float64 | USD | TLC Trip Record | tip_amount >= 0 & <= total_amount | Dibatasi (capping) / dibuang |
**total_amount** | float64 | USD | TLC Trip Record | total_amount > 0 | Baris dibuang (filtering) |
**tanggal_pickup** | object (date) | Tanggal (YYYY-MM-DD) | Ekstraksi Internal | Valid pada kalender 2023 | Terisi otomatis dari ekstraksi |
**nama_libur** | string | Teks | Nager.Date API | Sesuai nama hari libur nasional AS | Dimputasi dengan 'Bukan Libur' |
"""

with open("kamus_data.md", "w") as f:
    f.write(isi_kamus_data)

print("... Berkas 'kamus_data.md' berhasil dibuat! ...")

# 2. Verifikasi Skema Dataset Terintegrasi
print("\nVerifikasi Tipe Data pada DataFrame Akhir (df_terintegrasi):")
display(df_terintegrasi.dtypes)

--- Berkas 'kamus_data.md' berhasil dibuat! ---

Verifikasi Tipe Data pada DataFrame Akhir (df_terintegrasi):

0
tpep_pickup_datetime    datetime64[us]
trip_distance           float64
total_amount            float64
tanggal_pickup          object
tanggal_libur           object
nama_libur              object

dtype: object
```

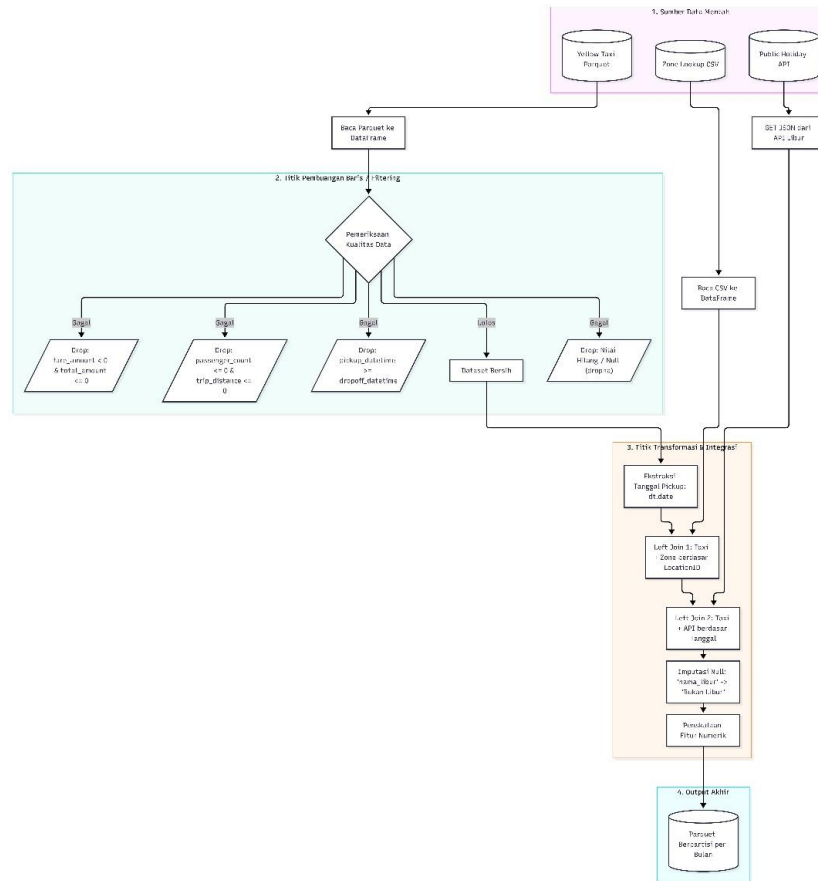
Kamus Data (Data Dictionary) - NYC Yellow Taxi Trip Data

Berikut adalah kamus data untuk dataset akhir hasil pipeline pra-pemrosesan dan penggabungan multi-sumber:

Nama Kolom	Tipe Data	Satuan	Sumber	Aturan Validitas	Penanganan Nilai Hilang
tpep_pickup_datetime	datetime64[ns]	Waktu (YYYY-MM-DD)	TLC Trip Record	Harus lebih kecil dari waktu dropoff	Baris dibuang (drop, dropna)
tpep_dropoff_datetime	datetime64[ns]	Waktu (YYYY-MM-DD)	TLC Trip Record	Harus lebih besar dari waktu pickup	Baris dibuang (drop, dropna)
passenger_count	float64	Orang	TLC Trip Record	passenger_count > 0	Baris dibuang (drop)
trip_distance	float64	Mil (Miles)	TLC Trip Record	trip_distance > 0	Baris dibuang (drop)
PULocationID	int64	ID Numerik	TLC Trip Record	Nilai 1 s.d. 265 (Sesuai Zone Lookup)	Baris dibuang jika lookup gagal
DOLocationID	int64	ID Numerik	TLC Trip Record	Nilai 1 s.d. 265 (Sesuai Zone Lookup)	Baris dibuang jika lookup gagal
Borough	string	Teks	TLC Zone Lookup	Tidak boleh bernilai "Unknown"	Baris dibuang (filtering)
Zone	string	Teks	TLC Zone Lookup	Sesuai daftar zona Kota New York	Diabaikan / Dibiarkan apa adanya
fare_amount	float64	USD	TLC Trip Record	fare_amount >= 0	Dikonversi ke absolut atau dibuang
tip_amount	float64	USD	TLC Trip Record	tip_amount >= 0 & <= total_amount	Dibatasi (capping) atau dibuang
total_amount	float64	USD	TLC Trip Record	total_amount > 0	Baris dibuang (drop)
tanggal_pickup	object (date)	Tanggal (YYYY-MM-DD)	Ekstraksi Internal	Valid pada kalender 2023	Terisi otomatis dari ekstraksi
nama_libur	string	Teks	Nager.Date API	Sesuai nama hari libur nasional AS	Dimputasi dengan string "Bukan Libur"

Dataset yang dihasilkan dari pipeline ini terdiri atas 13 fitur utama yang berasal dari tiga sumber data berbeda. Kamus data dibuat untuk menjelaskan struktur dataset secara terbuka, memastikan setiap data tetap konsisten dan berkualitas, serta menerangkan batasan penggunaan data yang valid. Selain itu, kamus tersebut juga menjelaskan cara nilai yang hilang ditangani selama proses ETL, mulai dari tahap pengambilan hingga pengolahan data.

5. Dokumen lineage. Buat diagram alur satu halaman (boleh gambar tangan yang difoto) dari sumber sampai dataset akhir, menandai setiap titik transformasi dan pembuangan baris.



Penjelasan Teknis Alur Data

- Pengambilan Data Awal

Tahap pertama dalam alur data adalah mengumpulkan tiga jenis sumber data yang memiliki format dan fungsi berbeda, yaitu:

- 1) NYC Yellow Taxi Trip Data (Parquet): Berisi jutaan catatan perjalanan taksi dan digunakan sebagai sumber data transaksi utama.
- 2) Taxi Zone Lookup Table (CSV): Digunakan untuk menerjemahkan kode LocationID pada lokasi penjemputan dan pengantaran menjadi nama wilayah, seperti Borough dan Zone.
- 3) Nager.Date Public Holiday API (JSON melalui HTTP GET): Merupakan layanan eksternal yang diakses melalui internet untuk memperoleh informasi mengenai hari libur nasional di Amerika Serikat.

- Tahap Penyaringan Data

Sebelum masuk ke proses transformasi, data perjalanan taksi terlebih dahulu diperiksa melalui beberapa aturan validasi kualitas, sebagaimana diterapkan

pada K-6 dan Latihan 2. Data yang tidak memenuhi ketentuan berikut akan dihapus:

- 1) Validasi operasional: Menghilangkan catatan dengan jumlah penumpang tidak valid, yaitu `passenger_count <= 0`, serta perjalanan yang tidak memiliki jarak tempuh, yaitu `trip_distance <= 0`.
- 2) Validasi urutan waktu: Menghapus data apabila waktu penjemputan terjadi pada atau setelah waktu pengantaran, dengan kondisi `tpep_pickup_datetime >= tpep_dropoff_datetime`.
- 3) Validasi nilai transaksi: Mengeluarkan catatan yang memiliki nilai tarif negatif (`fare_amount < 0`) atau total pembayaran nol maupun negatif (`total_amount <= 0`).
- 4) Pembersihan data kosong: Menghapus baris yang memiliki nilai Null atau NaN pada kolom-kolom penting menggunakan fungsi pembersihan standar.

- Tahap Transformasi dan Penggabungan

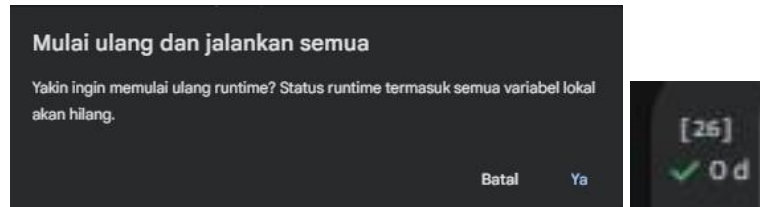
Setelah data yang tidak valid disaring, dataset kemudian diperkaya melalui beberapa proses berikut:

- 1) Membuat kolom tanggal. Bagian tanggal diambil dari `tpep_pickup_datetime` menggunakan `.dt.date`, lalu disimpan dalam kolom baru bernama `tanggal_pickup`.
- 2) Penggabungan data zona: Informasi perjalanan taksi dihubungkan dengan Taxi Zone Lookup melalui `PULocationID` dan `DOLocationID`. Proses ini bertujuan menambahkan nama wilayah asal dan tujuan perjalanan.
- 3) Penggabungan informasi hari libur: Dataset taksi kembali digabungkan dengan data dari `Nager.Date API`. Pencocokan dilakukan berdasarkan kesamaan antara `tanggal_pickup` dan `tanggal_libur`.
- 4) Pengisian data kosong: Nilai NaN yang muncul karena tidak adanya kecocokan pada data hari libur diganti dengan teks "Bukan Libur". Label tersebut menunjukkan bahwa perjalanan berlangsung pada hari operasional biasa.
- 5) Penyamaan skala fitur: Fitur numerik dinormalisasi menggunakan metode seperti `StandardScaler` atau `MinMaxScaler` agar siap digunakan dalam proses pemodelan.

- Penyimpanan dan Pembagian Data

- 1) Dataset yang telah selesai diproses disimpan dalam format Parquet berpartisi pada direktori penyimpanan, dengan `bulan_eksekusi` digunakan sebagai dasar pembagian data.

- 2) Dengan cara ini, data dari setiap bulan tersimpan secara terpisah. Struktur tersebut membantu mencegah terjadinya duplikasi data maupun penimpanan catatan dari bulan yang berbeda.
6. Reproducibility, notebook harus dapat dijalankan kembali dari awal menggunakan runtime yang masih bersih melalui opsi Restart and Run All. Jika pipeline mengalami kegagalan saat diulang, nilai untuk aspek terkait dibatasi hingga maksimal 60.



Berdasarkan pengujian yang dilakukan, notebook ini berhasil melewati proses Restart and Run All tanpa kendala pada runtime yang bersih.

f. Pertanyaan Evaluasi

1. Mengapa NoSQL sering dibaca “Not Only SQL”? Sebutkan dua situasi ketika tabel relasional tetap pilihan yang lebih baik.

Jawab

NoSQL sering dibaca “Not Only SQL” karena NoSQL bukan pengganti SQL atau tabel relasional, melainkan merupakan pilihan tambahan untuk kebutuhan penyimpanan dan pengolahan data tertentu. NoSQL digunakan ketika data memiliki bentuk yang beragam, berubah-ubah, berukuran sangat besar, atau ketika terdapat pertanyaan yang kurang nyaman dilakukan menggunakan tabel. Dalam sistem nyata, beberapa jenis penyimpanan bahkan dapat digunakan secara bersamaan sesuai kebutuhan.

Tabel relasional tetap lebih sesuai, misalnya:

1. Transaksi pembayaran yang membutuhkan konsistensi data dan transaksi yang terjamin.
2. Pengolahan data terstruktur dengan JOIN dan agregasi, ketika hubungan antar-data dan query SQL lebih mudah dikelola menggunakan tabel relasional.

2. Apa perbedaan BASE dan ACID? Jelaskan teorema CAP dengan satu contoh data pada aplikasi taksi yang sebaiknya memilih consistency, dan satu yang boleh memilih availability.

Jawab

BASE lebih mengutamakan ketersediaan dan kecepatan sehingga data dapat bersifat eventually consistent, artinya untuk sementara beberapa pembaca

mungkin melihat data yang belum terbaru. ACID lebih menekankan keandalan transaksi sehingga perubahan data harus berlangsung secara aman dan konsisten. Modul menjelaskan bahwa Delta Lake membawa keandalan tingkat ACID, termasuk penulisan yang bersifat atomik.

Teorema CAP menjelaskan bahwa ketika terjadi gangguan atau pemutusan jaringan antar-mesin, sistem harus memilih antara consistency atau availability.

- Consistency, pada data saldo pembayaran atau transaksi pembayaran taksi, consistency lebih penting karena data yang dibaca harus menunjukkan keadaan terbaru dan tidak boleh terjadi perbedaan saldo.
- Availability, pada informasi daftar perjalanan terbaru, availability dapat lebih diutamakan karena daftar tersebut masih dapat ditampilkan meskipun isinya sedikit terlambat diperbarui.

3. Mengapa pencarian satu data pada key-value sangat cepat, tetapi menghitung sangat lambat? Apa yang dikorbankan key-value demi kesederhanaannya?

Jawab

Key-value sangat cepat untuk pencarian satu data karena data langsung diakses menggunakan key, seperti loker yang dapat dibuka langsung berdasarkan nomor. Sistem tidak perlu memeriksa isi semua data. Sebaliknya, untuk menghitung atau mencari berdasarkan isi, setiap nilai harus dibuka dan diperiksa terlebih dahulu. Akibatnya, proses menjadi lebih lambat. Yang dikorbankan adalah kemampuan melakukan pencarian dan agregasi berdasarkan isi data demi struktur yang sederhana dan akses langsung melalui key.

Pada percobaan modul, key-value sekitar 7 kali lebih lambat daripada SQL untuk perhitungan rata-rata karena seluruh nilai harus dibuka terlebih dahulu.

4. Jelaskan perbedaan menyatukan data (embedding) dan merujuk (referencing) pada model document, dan satu kondisi yang membuat masing-masing lebih tepat.

Jawab

- Embedding adalah menyatukan data yang saling berkaitan ke dalam satu dokumen. Keuntungannya adalah data dapat dibaca lebih cepat dan dalam satu kali akses, tetapi konsekuensinya data yang sama dapat mengalami duplikasi.
- Referencing adalah menyimpan data secara terpisah dan menyimpan ID atau referensi untuk menghubungkannya. Cara ini lebih hemat penyimpanan dan mengurangi duplikasi, tetapi membutuhkan langkah tambahan untuk mengambil data terkait.

- Embedding lebih tepat ketika data terkait sering dibaca secara bersamaan dan jarang berubah. Referencing lebih tepat ketika data yang sama digunakan oleh banyak dokumen atau sering diperbarui secara terpisah, sehingga duplikasi dapat dikurangi.

5. Kapan graph lebih unggul dari SQL untuk pertanyaan keterhubungan? Berikan alasan yang tidak bergantung pada kecepatan.

Jawab

Graph lebih sesuai ketika pertanyaan berfokus pada hubungan, jalur, atau keterjangkauan antar-entitas, misalnya mencari zona yang dapat dicapai dalam beberapa langkah perjalanan, sistem rekomendasi, atau hubungan antar-node.

Alasannya bukan semata-mata karena graph lebih cepat, tetapi karena hubungan antar-data direpresentasikan secara langsung sebagai node dan edge. Pada graph, pertanyaan seperti “dalam paling banyak k langkah” dapat dilakukan melalui traversal jaringan. Pada SQL, pertanyaan dengan banyak tingkat hubungan memerlukan JOIN tambahan untuk setiap langkah.

Dalam percobaan modul, graph dan SQL menghasilkan jawaban yang sama, sehingga modul menekankan bahwa keunggulan graph terletak pada cara memodelkan dan menanyakan hubungan, bukan berarti graph selalu lebih cepat.

6. Apa yang dilakukan indeks spasial sebelum memeriksa lokasi secara tepat? Mengapa klaim “indeks 1.000 kali lebih cepat” bisa menyesatkan?

Jawab

Indeks spasial seperti R-tree atau STRtree terlebih dahulu mengelompokkan titik-titik yang berdekatan ke dalam kotak atau wilayah tertentu. Ketika pencarian dilakukan, kelompok yang batas kotaknya tidak beririsan dengan wilayah yang dicari dapat langsung dilewati. Setelah kandidat ditemukan, barulah pemeriksaan lokasi secara tepat dilakukan. Cara ini mirip dengan partition pruning, tetapi berdasarkan posisi geografis.

Klaim “1.000 kali lebih cepat” dapat menyesatkan karena hasil perbandingan sangat bergantung pada pembanding yang digunakan. Dalam percobaan modul, STRtree sekitar ribuan kali lebih cepat dibanding loop Python, tetapi jika dibandingkan dengan NumPy, kecepatannya hanya sekitar 2 kali lebih cepat. Jadi angka percepatan harus selalu menyebutkan metode pembandingnya.

7. Apa isi folder `_delta_log/` dan bagaimana ia membuat penulisan bersifat atomik dan time travel mungkin? Mengapa VACUUM dengan retention 0 jam berbahaya?

Jawab

- Folder `_delta_log/` berisi transaction log yang mencatat setiap perubahan pada tabel Delta. Setiap perubahan tabel dicatat dalam berkas log, termasuk file yang ditambahkan atau dikeluarkan dari tabel. Keadaan tabel pada suatu versi diperoleh dengan membaca catatan transaksi sampai versi tersebut.
- Mekanisme tersebut membuat penulisan bersifat atomik, karena pembaca melihat kondisi tabel sebelum atau sesudah suatu commit, bukan kondisi setengah jadi. Mekanisme ini juga memungkinkan time travel, karena versi lama masih dapat dibentuk kembali dari catatan transaksi dan file Parquet yang masih tersedia.
- VACUUM dengan retention 0 jam berbahaya karena dapat langsung menghapus file lama yang sebenarnya masih diperlukan untuk membaca versi sebelumnya. Akibatnya, time travel ke versi lama dapat terputus, dan proses lain yang masih menggunakan file tersebut juga dapat terganggu. Modul secara khusus memperingatkan agar retention 0 jam tidak digunakan pada tabel sungguhan; contoh tersebut hanya digunakan dengan `dry_run=True` untuk melihat file yang akan dihapus.

D. Kesimpulan

Berdasarkan seluruh rangkaian praktikum, proses prapemrosesan berhasil mengubah data mentah perjalanan taksi New York menjadi dataset yang lebih terstruktur, bersih, dan informatif. Tiga sumber data catatan perjalanan, informasi zona wilayah, serta data hari libur nasional berhasil digabungkan dengan baik melalui metode left join. Penggunaan imputasi yang sesuai membantu melengkapi informasi tanpa mengubah susunan utama dataset. Selain itu, dokumen data lineage telah mencatat secara jelas setiap tahapan penghapusan data dan perubahan yang dilakukan, sehingga proses pembentukan dataset akhir dapat ditelusuri dan diperiksa kembali.

Pada tahap pengujian, notebook berhasil dijalankan ulang menggunakan fitur Restart and Run All pada runtime yang masih bersih tanpa mengalami kesalahan maupun masalah alur logika. Hasil tersebut menunjukkan bahwa pipeline memiliki sifat idempotent, yaitu dapat dieksekusi berkali-kali tanpa menimbulkan data ganda atau kerusakan pada hasil pemrosesan. Dataset yang telah terintegrasi kemudian disimpan dalam format Parquet berpartisi, sehingga lebih siap digunakan untuk analisis Big Data pada tahap berikutnya.

Link Github:

https://github.com/Aramadhanny/BigData26_A_2411533016_Abdullah-Al-Ramadhani

Link Drive:

https://drive.google.com/drive/folders/14dtGZHDLU_jVJhlCD1TE0sS_Yt8J3wck?usp=sharing